

CS516: Parallelization of Programs

Breadth First Search in GPU

Vishwesh Jatala

Assistant Professor

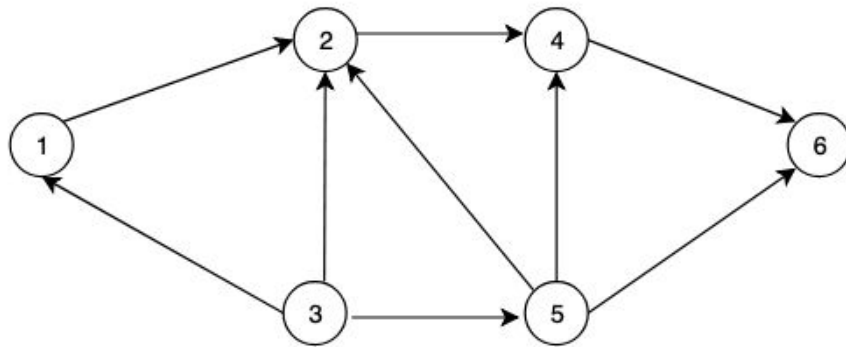
Department of CSE

Indian Institute of Technology Bhilai

vishwesh@iitbhilai.ac.in



Graph Representation

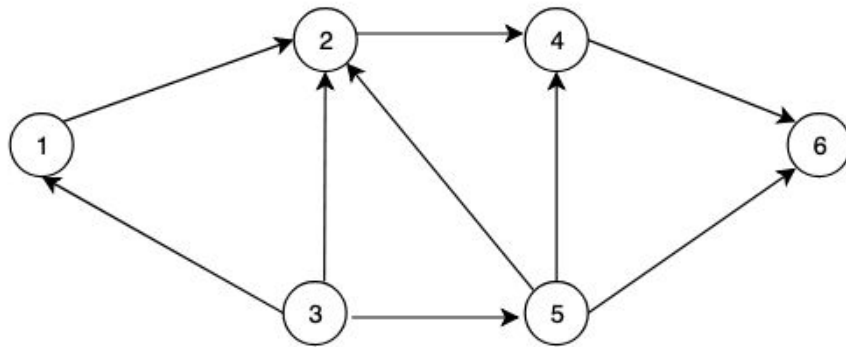


Adjacency Matrix

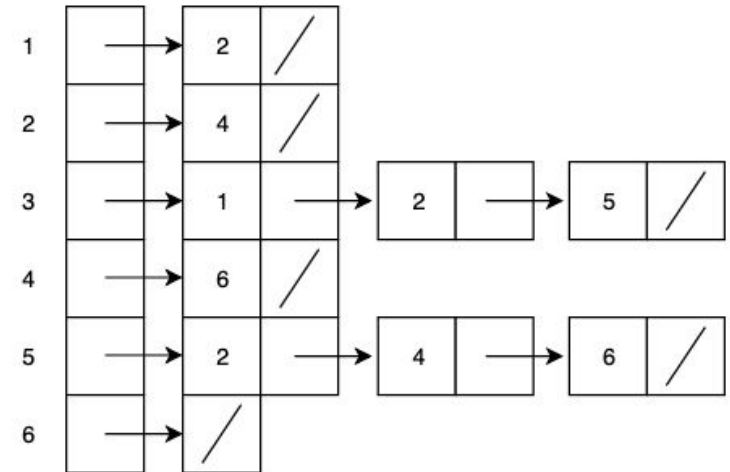
	1	2	3	4	5	6
1	0	1	0	0	0	0
2	0	0	0	1	0	0
3	1	1	0	0	1	0
4	0	0	0	0	0	1
5	0	1	0	1	0	1
6	0	0	0	0	0	0

- If an edge $u \rightarrow v$ is present, the $A[u][v] = 1$
- Useful for **dense** matrices
- Storage: $O(|V|^2)$

Graph Representation

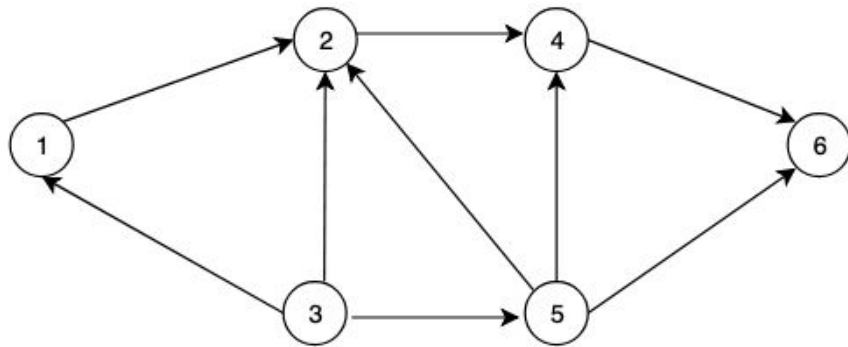


Adjacency List



- For each vertex u , a list is created for maintaining neighbors
- Useful for **sparse** matrices
- Storage: $O(|V|+|E|)$

Graph Representation

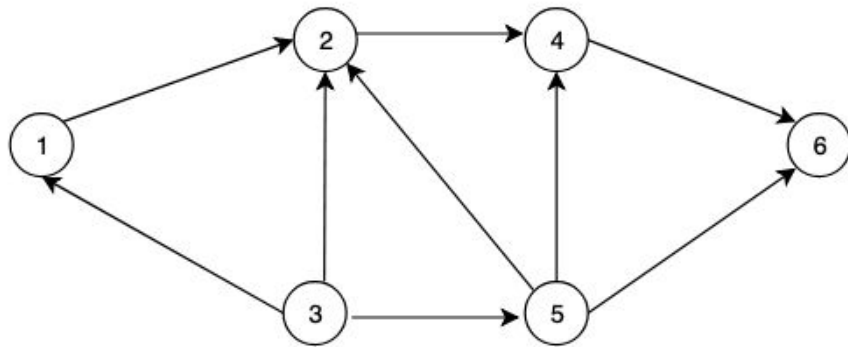


Coordinated List

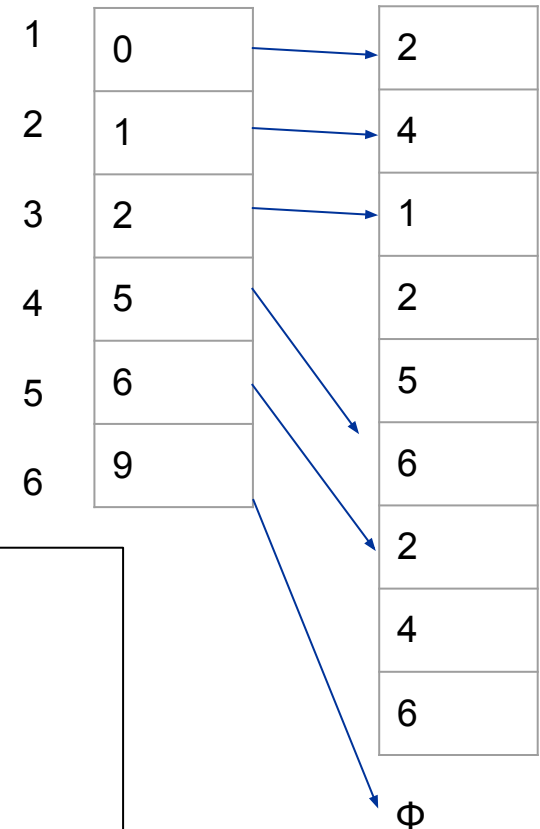
1	2
2	4
3	1
3	2
3	5
4	6
5	2
5	4
5	6

- Storage $O(|E|)$
- Useful for edge centric algorithms:
 - Triangle counting, ktruss decomposition, etc.

Graph Representation



Compressed Sparse Row



- Similar to adjacent lists, but in arrays
- $O(|E|+|V|)$
- Useful for Sparse graphs
- **Widely used!**

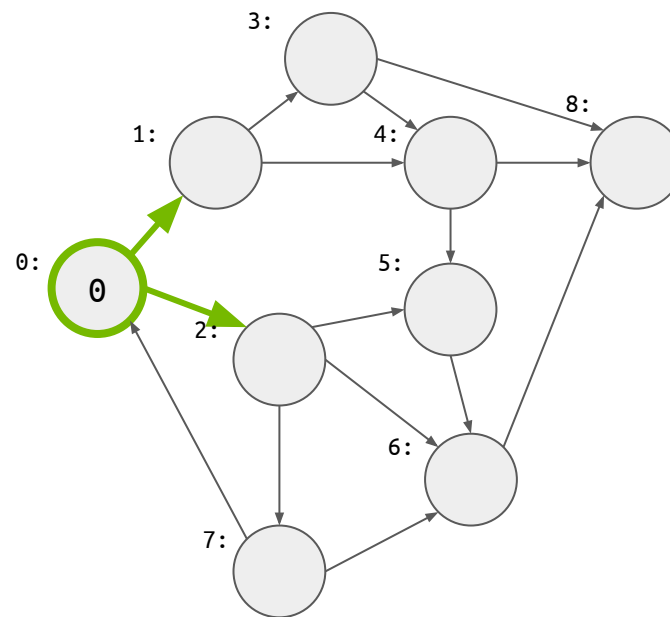
Breadth-First Search

Problem:

Given a source node S , find the number of steps required to reach each node N in the graph

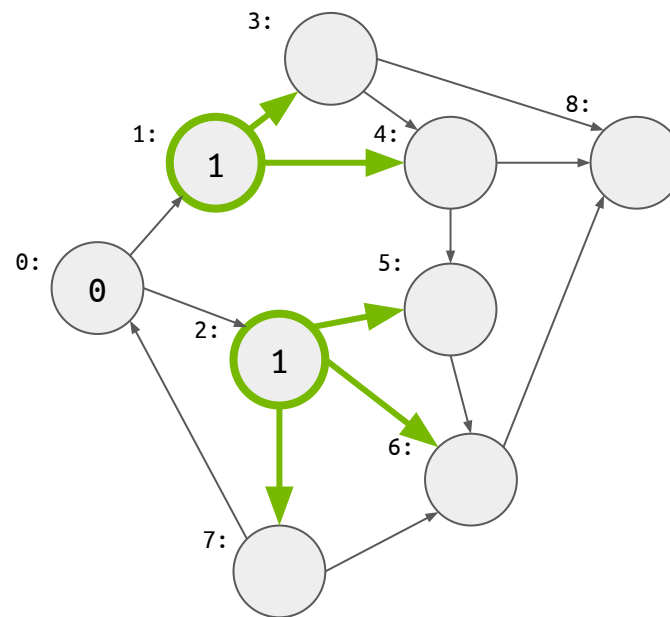
Given this labelling of the graph, one can easily find a shortest path from S to a destination T

Breadth First Search



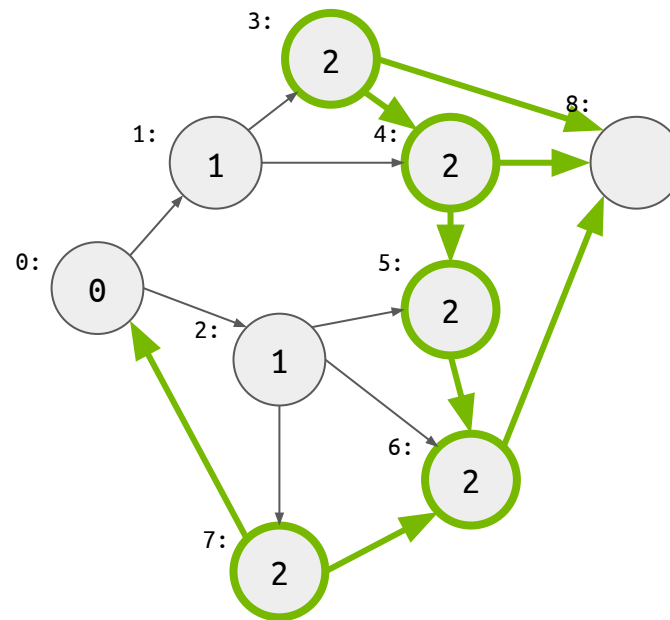
Source = 0
Round 0

Breadth First Search



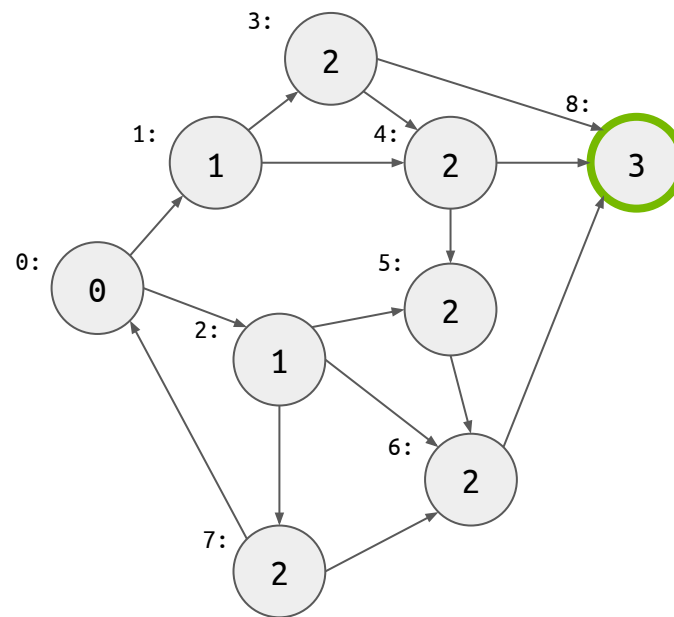
Source = 0
Round 1

Breadth First Search



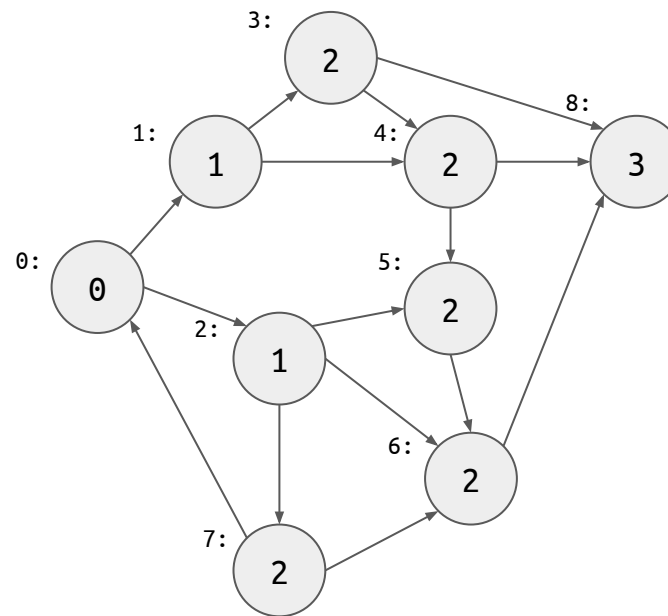
Source = 0
Round 2

Breadth First Search



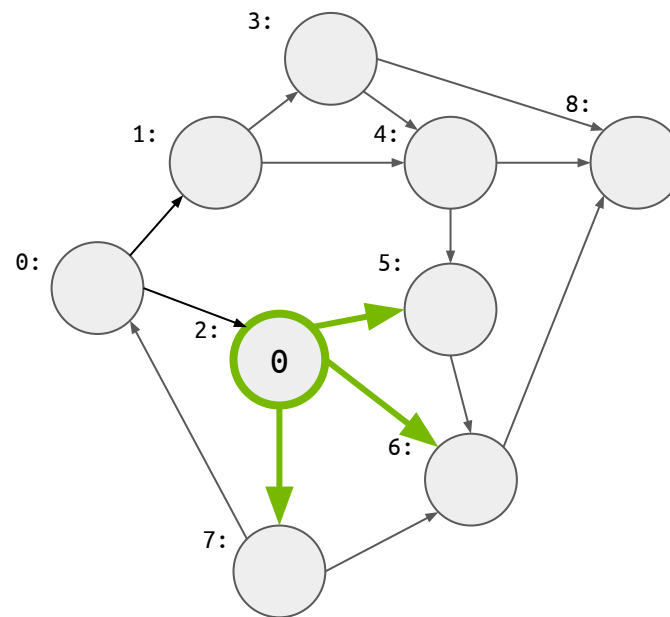
Source = 0
Round 3

Breadth First Search



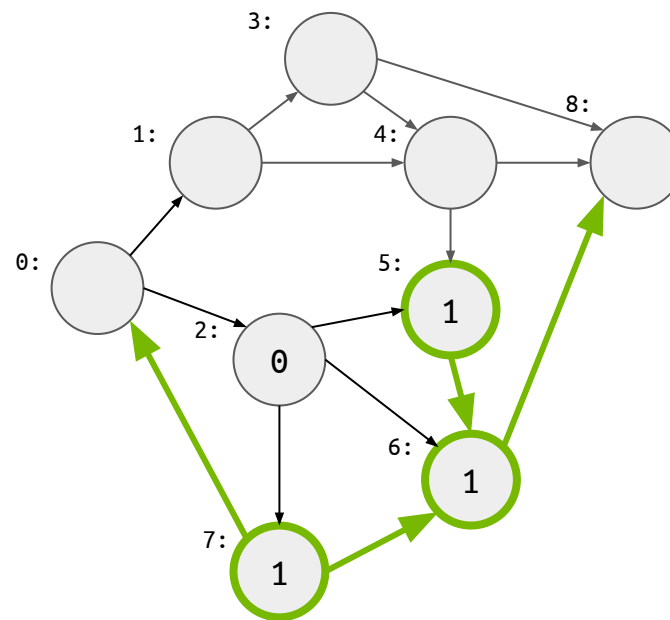
Source = 0

Breadth First Search



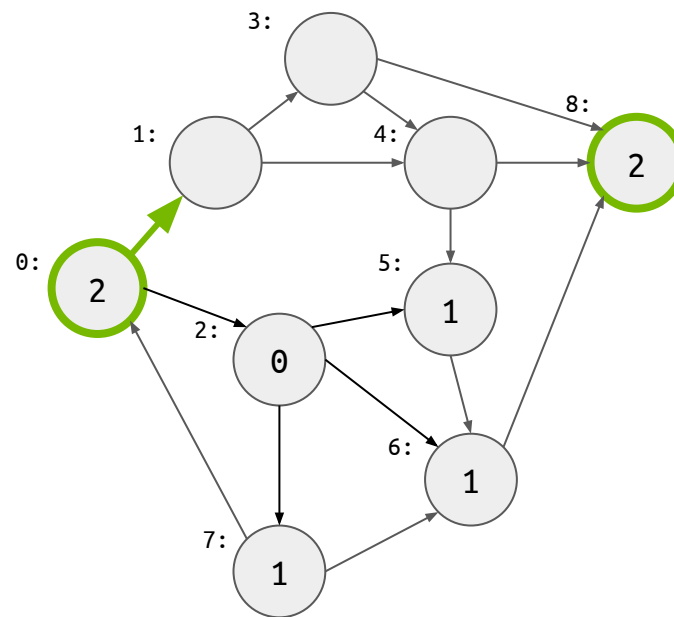
Source = 2
Round 0

Breadth First Search



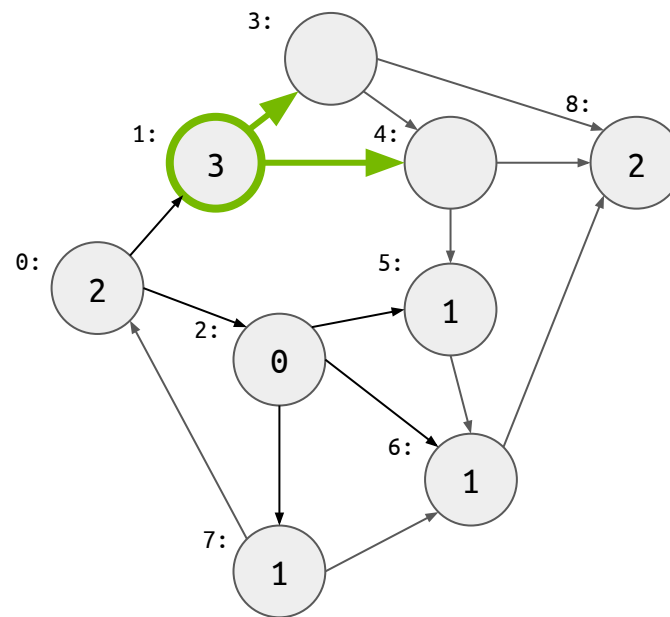
Source = 2
Round 1

Breadth First Search



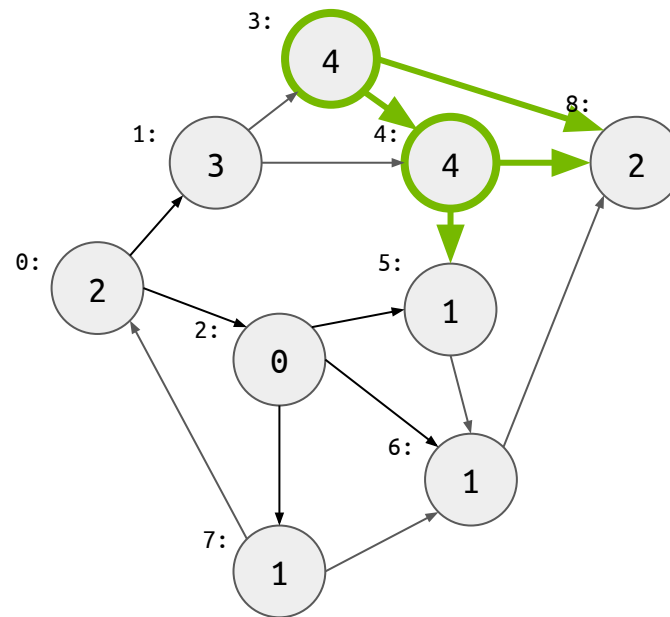
Source = 2
Round 2

Breadth First Search



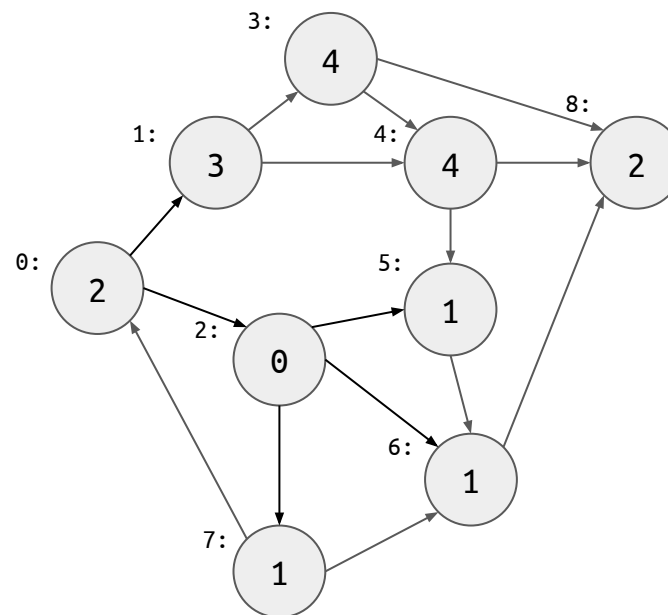
Source = 2
Round 3

Breadth First Search



Source = 2
Round 4

Breadth First Search



Source = 2

Sequential BFS

Most computer scientists are familiar with C++ / Java / Python / etc - style BFS implementations using language - provided data structures (e.g. queue)

We'll look at a C-style implementation to ease the translation into CUDA

Sequential BFS

```
void BFS_sequential(int source, const int * rowPointers, const int * destinations, int * distances) {

    int frontier[2][MAX_FRONTIER_SIZE];
    int * currentFrontier = &frontier[0];
    int currentFrontierSize = 0;
    int * previousFrontier = &frontier[1];
    int previousFrontierSize = 0;

    insertIntoFrontier(source, previousFrontier, &previousFrontierSize);
    distances[source] = 0;

    while (previousFrontierSize > 0) {
        // visit all vertices on the previous frontier
        for (int f = 0; f < previousFrontierSize; f++) {
            const int currentVertex = previousFrontier[f];
            // check all outgoing edges
            for (int i = rowPointers[currentVertex]; i < rowPointers[currentVertex+1]; ++i) {
                if (distances[destinations[i]] == -1) {
                    // this vertex has not been visited yet
                    insertIntoFrontier(destinations[i], currentFrontier, &currentFrontierSize);
                    distances[destinations[i]] = distances[currentVertex] + 1;
                }
            }
        }
        swap(currentFrontier, previousFrontier);
        previousFrontierSize = currentFrontierSize;
        currentFrontierSize = 0;
    }
}
```



In practice, we'd want to check for and handle overflow here

Sequential BFS

```
void insertIntoFrontier(int vertex, int * frontier, int * frontierSize) {  
    frontier[*frontierSize] = vertex;  
    ++(*frontierSize);  
}
```

One Parallelization Approach

- Assign one thread per vertex
- For each iteration, check all incoming edges to see if the source vertex was just visited in the last iteration; if so, mark as visited in this iteration
- Not very work efficient; $O(VL)$ for V = number of vertices, L = length of longest path
- Difficult to detect stopping criterion

A More Work-Efficient Approach

- Parallelize each individual iteration of the while loop in the sequential BFS code
- Assign a section of the vertices in the previous frontier to each thread
- Introduce a synchronization point at the end of each iteration

Parallel BFS Host Code

```
void BFS_host(int source, const int * rowPointers, const int * destinations, int * distances) {

    int dFrontier[2][MAX_FRONTIER_SIZE];
    int * dCurrentFrontierSize;
    int * dPreviousFrontierSize; int hPreviousFrontierSize;
    int * dVisited;

    int * dCurrentFrontier = &frontier[0];
    int * dPreviousFrontier = &frontier[1];

    // allocate device memory, copy memory from device to host, initialize frontier sizes, etc.
    ...

    hPreviousFrontierSize = 1;

    while (hPreviousFrontierSize > 0) {
        int numBlocks = (hPreviousFrontierSize-1) / BLOCK_SIZE + 1;

        BFS_Bqueue_kernel<<<numBlocks, BLOCK_SIZE>>>(dPreviousFrontier, dPreviousFrontierSize,
                                                    dCurrentFrontier, dCurrentFrontierSize,
                                                    dRowPointers, dDestinations, dDistances, dVisited);

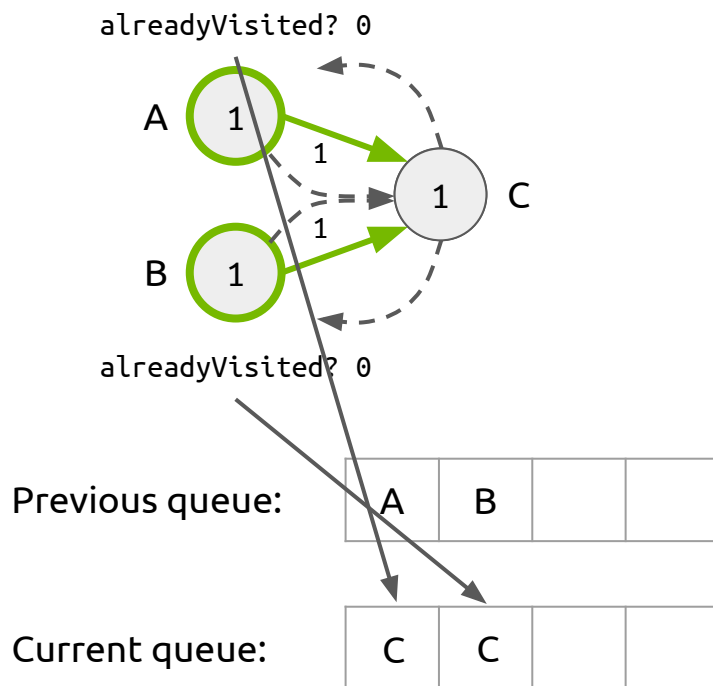
        swap(dCurrentFrontier, dPreviousFrontier);
        cudaMemcpy(dPreviousFrontierSize, dCurrentFrontierSize, sizeof(int), cudaMemcpyDeviceToDevice);
        cudaMemcpy(dCurrentFrontierSize, 0, sizeof(int));

        cudaMemcpy(&hPreviousFrontierSize, dPreviousFrontierSize, sizeof(int), cudaMemcpyDeviceToHost);
    }
}
```

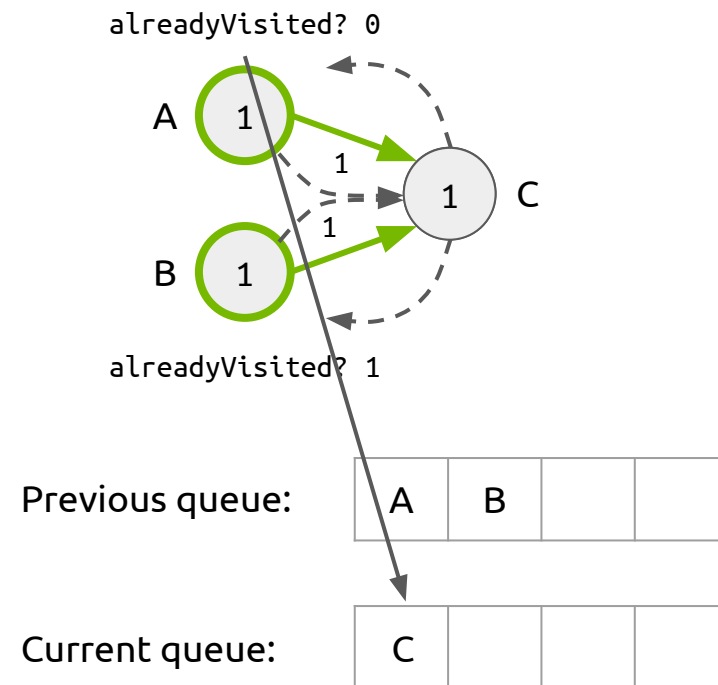
Output Interference in BFS

- We'll use flags to mark whether or not a vertex has been visited
- From a correctness perspective, output interference on flags can be ignored
- However, this will lead to additional work

Without synchronization:



With atomicExch:



Basic Parallel BFS Kernel

```
__global__ void BFS_Queue_kernel(const int * previousFrontier, const int * previousFrontierSize,
                                int * currentFrontier, int * currentFrontierSize, const int * rowPointers,
                                const int * destinations, int * distances, int * visited) {

    const int t = threadIdx.x + blockDim.x * blockIdx.x;
    if (t < *previousFrontierSize) {

        const int vertex = previousFrontier[t];
        for (int i = rowPointers[vertex]; i < rowPointers[vertex+1]; ++i) {

            // check visitation atomically, avoiding redundant expansion
            const int alreadyVisited = atomicExch(&(visited[destinations[i]]),1);
            if (!alreadyVisited) {

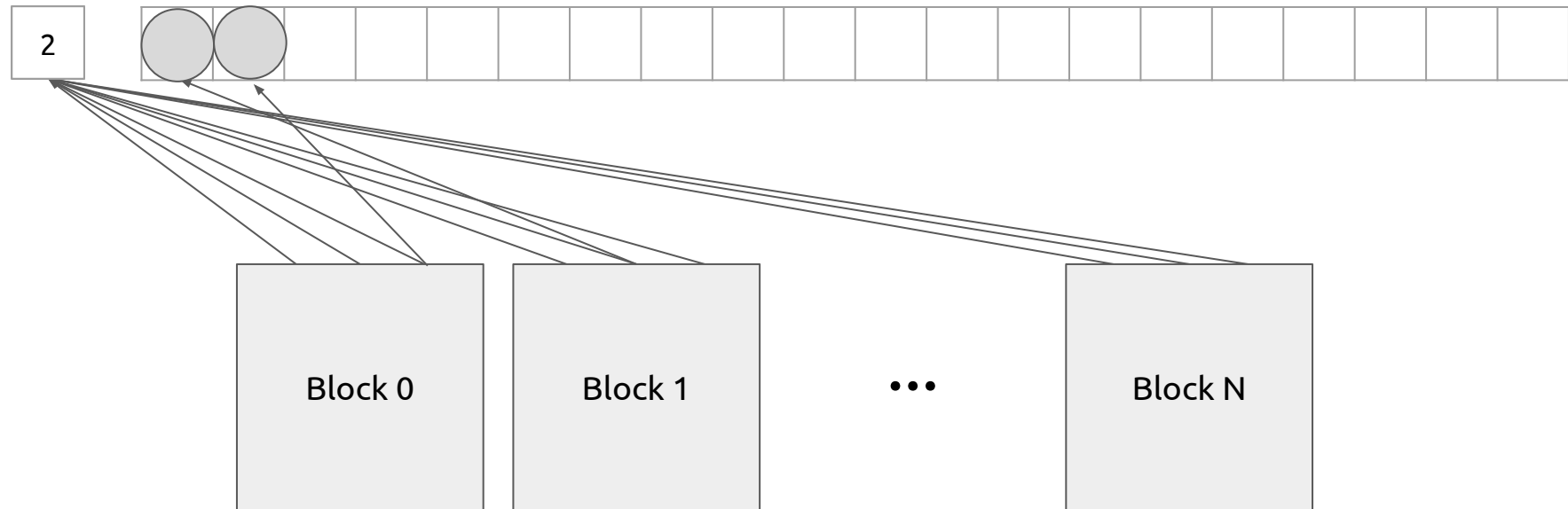
                // we're visiting a new vertex: get a spot in line atomically
                const int queueIndex = atomicAdd(currentFrontierSize, 1);

                // place the vertex in line
                currentFrontier[queueIndex] = destinations[i];

            }
        }
    }
    __syncthreads();
}
```

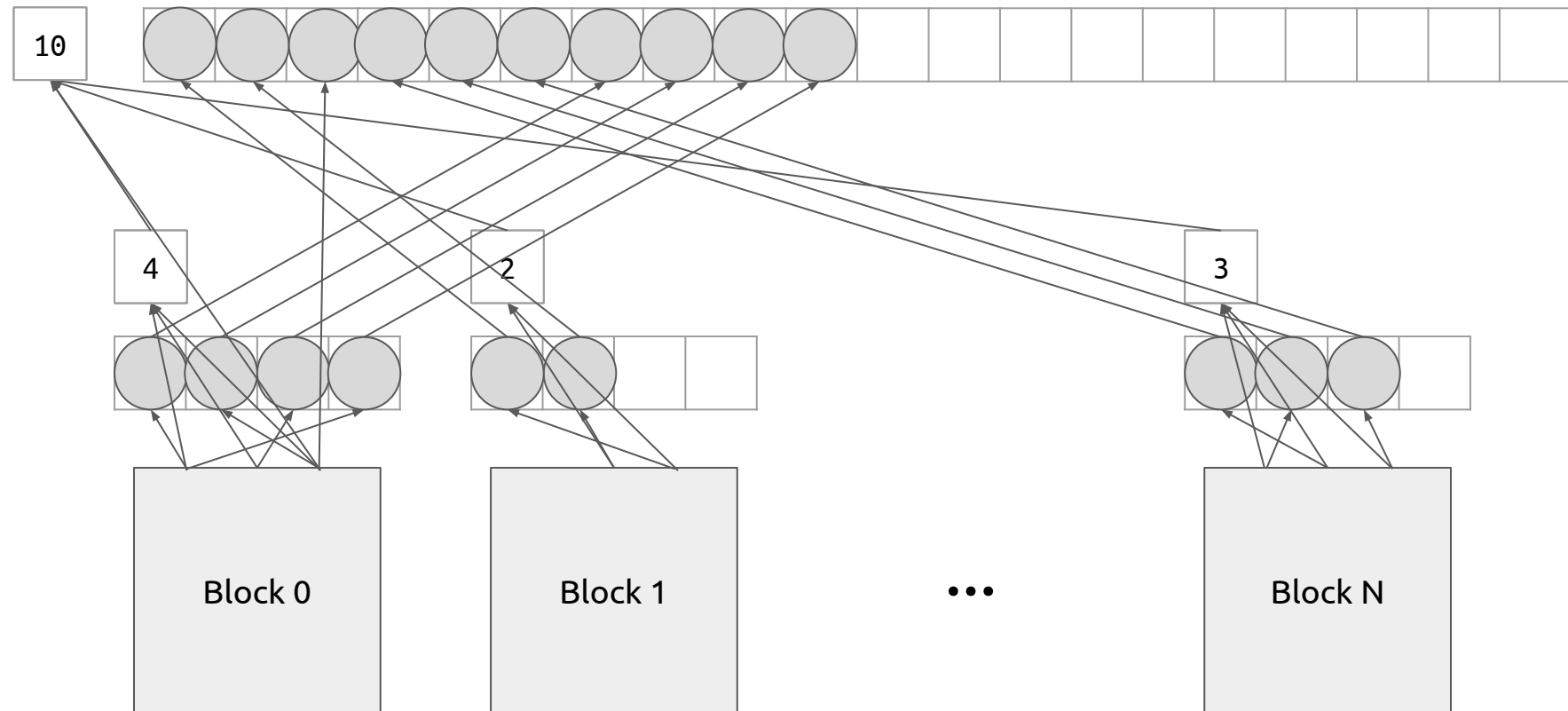
Output Interference in Frontier Queue

- There is also output interference in placing vertices in the queue
- Synchronization is strictly required here for correct output
- This is a bottleneck of the basic kernel



Privatization of the Frontier Queue

- We can make a private, block-level copy of the frontier queue
- Once complete, the private queues are combined to form the global queue



Parallel BFS Kernel with Privatization

```
__global__ void BFS_Bqueue_kernel(const int * previousFrontier, const int * previousFrontierSize,
                                  int * currentFrontier, int * currentFrontierSize, const int * rowPointers,
                                  const int * destinations, int * distances, int * visited) {

    __shared__ int sharedCurrentFrontier[BLOCK_QUEUE_SIZE];
    __shared__ int sharedCurrentFrontierSize, blockGlobalQueueIndex;

    if (threadIdx.x == 0) sharedCurrentFrontierSize = 0;
    __syncthreads();

    const int t = threadIdx.x + blockDim.x * blockIdx.x;
    if (t < *previousFrontierSize) {
        const int vertex = previousFrontier[t];
        for (int i = rowPointers[vertex]; i < rowPointers[vertex+1]; ++i) {
            const int alreadyVisited = atomicExch(&(visited[destinations[i]]),1);
            if (!alreadyVisited) {
                distances[destinations[i]] = distances[i] + 1;
                const int sharedQueueIndex = atomicAdd(&sharedCurrentFrontierSize,1);
                if (sharedQueueIndex < BLOCK_QUEUE_SIZE) { // there is space in the local queue
                    sharedCurrentFrontier[sharedQueueIndex] = destinations[i];
                } else { // go directly to the global queue
                    sharedCurrentFrontierSize = BLOCK_QUEUE_SIZE;
                    const int globalQueueIndex = atomicAdd(currentFrontierSize, 1);
                    currentFrontier[globalQueueIndex] = destinations[i];
                }
            }
        }
    }
    __syncthreads();

    if (threadIdx.x == 0) blockGlobalQueueIndex = atomicAdd(currentFrontierSize, sharedCurrentFrontierSize);
    __syncthreads();

    for (int i = threadIdx.x; i < sharedCurrentFrontierSize; i += blockDim.x) {
        currentFrontier[blockGlobalQueueIndex + i] = sharedCurrentFrontier[i];
    }
}
```